

COPADO | DESTRUCTIVE CHANGES BEST PRACTICES

KB0099391 - [Latest Version](#)

☆☆☆☆☆ 2 views

This is a brief walkthrough of the recommended steps of doing destructive change within the Copado environments. A destructive change is a user story that will remove the component in both the repo and the git branches.

When to use it

Like everything in the pipeline, all changes need to be promoted by Copado. Since many of the metadata is shared by teams, your changes need to be acknowledged by different teams. Below are a few scenarios (keep in mind that, you will also have the CO and User Stories already made before with the specific release):

- **Team needs to delete a method in a class:** This can be done via the regular metadata commit, a destructive change is not needed.
- **Team needs to disable a trigger / rule:** This can be done via the regular metadata commit, a destructive change is not needed.
- **Team needs to remove a profile:** This needs to be done via a destructive change, but you will need approval from the Security Team as the existing user needs to be updated.
- **Team needs to delete a field in the object:** This needs to be performed via destructive change. However, you will have to consider exporting the data before the change, and ensure there are no dependencies from other teams.
- **Team needs to delete an object (or a class, app) named abcd.xxx, the developer confirmed that there are no data and no dependencies:** Although it may look like there are dependencies, this field might already be used by other teams or they are basing their development on this field. You will need to inform the other team before the change, and ensure there are no dependencies / in-progress work.
- **Team need to remove the data that were incorrectly loaded into the prod environment:** This is not a metadata change, Copado is not needed.
- **Team need to remove an already deleted metadata that accidentally got committed into the git repo (no changes on SF, not linked to any stories):** This can be done via the destructive snapshot commit, user story is not needed.

How to make a destructive change

Copado has a detailed step by step instruction, [Destructive Changes](https://docs.copado.com/article/o3atav179r-destructive-changes) (<https://docs.copado.com/article/o3atav179r-destructive-changes>) (please skip step 4).

Why skip step 4? - It won't be applied if you don't click Apply. Please review the link in detail to get an overall understanding of the destructive change. Below are the recommended practices.

Note:

More explanation on, If the developer should delete the metadata from the org first and then manually add the api name on the commit, or if they should commit first and then delete the metadata on the org once the change is completed?

There's really no right or wrong way to do this, but if the developer gets into the habit of adding components to the User Story first, it's easy to select the components from the Commit Grid without having to select a different Credential. After the components have been committed, the developer should then remove the components from the source sandbox. But, sometimes the component has been removed prematurely. As the article mentions, the developer can select a different Credential in order to refresh the Grid to find the component name from another sandbox or can use the Add Row button to enter the Developer Name (API Name) manually. Keep in mind that cleaning up the components on the source sandbox must be done by hand, but will be removed automatically for sandboxes which the User Story is promoted/back-promoted to.

Choosing Base Branch and the Source Environment

If you are not sure about what the base branch is, please do not perform this activity.

Before deleting the component, you will need to know where the component is located.

- If your target deletion environment is PROD, you should always use the main branch as base branch and start from your dev environment. This is the default option.
- This is a valid option. I would include it in your document since it is a good way to resolve this edge-case scenario. If you are deleting one single component that has not been moved to prod, and this change is only in one sandbox, and this single component is the only metadata that needs to be deleted in the whole release. You should choose the final destination as the base branch. The story will have the source environment (credential) as the sandbox immediately below. Make sure NOT to promote it beyond the final target environment by marking the story as cancelled and check the "Exclude from CBM" and "Stop Indexing" checkbox.
- Recommended approach: If you're deleting components that are not in production, you need to find the whole user story and you must create the commit base of the feature branch from that specific user story. Make sure NOT to promote it beyond the final target environment by marking the story as cancelled and check the "Exclude from CBM" and "Stop Indexing" checkbox once it reaches the destination branch. Warning: If you use the main branch, in this case, and there is no warning "No Changes to Commit" after you commit, everything will appear to be working. In fact, when you promote the story, the components will be removed from the target org, but they will not be removed from the target org branch. Now you will have a mess to clean-up later in Git. Better to do it right up-front.

Note: make sure to click on "save" after updating the base branch.

Apex Code Coverage	
Minimum Apex Code Coverage ⓘ	Has Apex Code ⓘ
85%	☒
Apex Code Coverage ⓘ	Classes Without Coverage ⓘ
89%	0
Failing Methods ⓘ	Triggers Without Coverage ⓘ
0	0

Developers only:

- If you need to remove an incorrect file in your user story that you have committed while the story is still in the dev environments, you can go ahead and click on re-commit, check "re-create feature branch" and select the needed metadata. Your feature branch will be recreated with the selected metadata and all the previous committed components would be deleted. Keep in mind that this is the only way to remove previously committed items, but it only works for additive/constructive commits. There is no option to recommit for a destructive change!***

*** This is a VERY important concept to understand because as soon as you commit a destructive change to the User Story feature branch, the branch is immediately merged into the org branch associated with the assigned Credential on the User Story. So, the destructive change occurs immediately in Git. As you can imagine, it's really hard to undo without some Git magic directly on the repo. For this reason, NEVER create a destructive change story directly in PROD because it will modify your main branch without you even deploying anything.

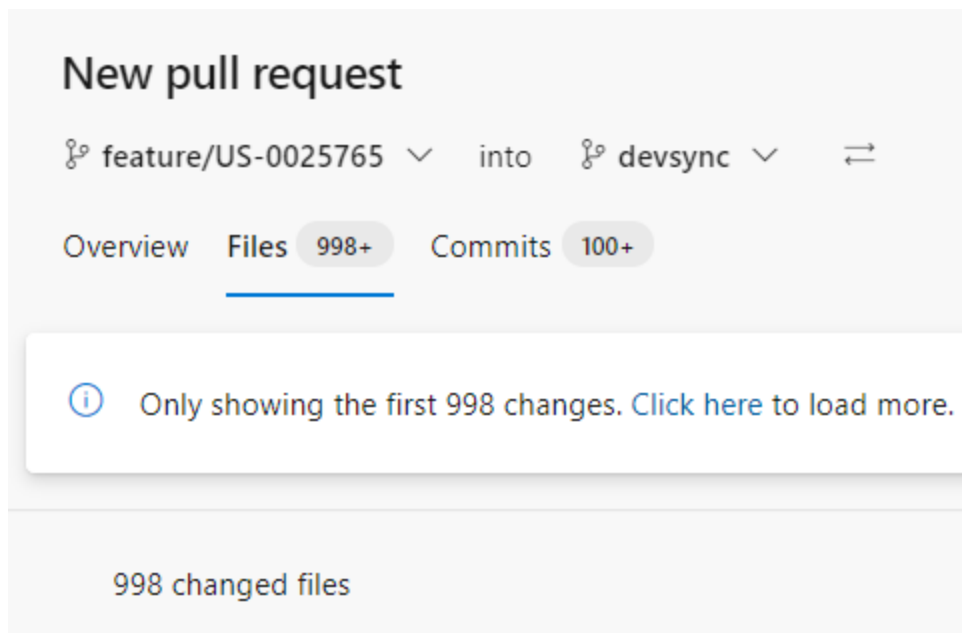
Also, do not refer to this other article about various destructive change scenarios. The first example is ok, but the others are bad/incomplete/incoherent. Don't choose the QA/UAT branch as you already know what can happen.

- If you need to make corrections for an incorrect Destructive Changes Commit:
 - Cancel the original User Story with the Destructive Change that was made in error
 - Status = Cancelled
 - Stop Indexing Metadata = True
 - Exclude from CBM = True
 - Create a new User Story with the same Credential that your original User Story started on, and create the correct/clean Destructive Change with all of the Components you wish to destroy
 - This Story can be promoted/deployed up the Pipeline as normal
 - Just make sure, once you promoted/deploy the Story the first time, to go back to the Source Salesforce Sandbox and manually delete the Components - the Destructive Change Commit will handle the Salesforce deletions as you go up, but it does not do the Salesforce delete in the Source Salesforce Sandbox
 - Last but not least, you'll need help from E3 to examine the source org branch and determine if there is anything that needs to be replaced (i.e. restored because something was included that you now no longer wish to include)
 - Once identified, you can get the proper code by going back (in time) a commit or two, grab the removed items and get them recommitted and back into the source branch

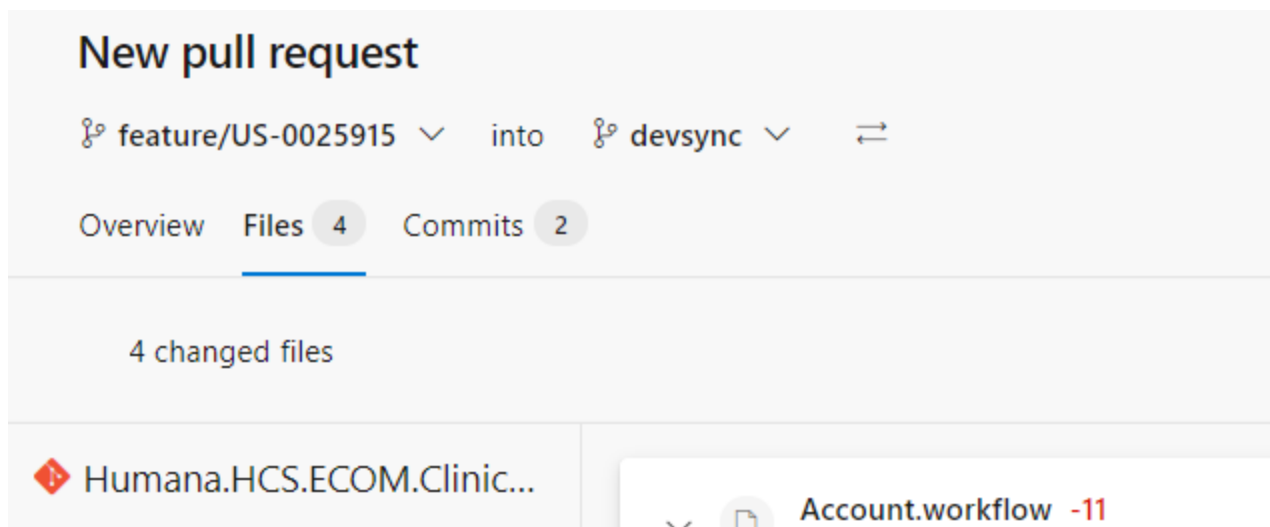
Perform Quality Checking is Required

Please always follow the E3 document [Best Practices \(https://humanaprod.service-now.com/kb_view.do?sysparm_article=KB0099390\)](https://humanaprod.service-now.com/kb_view.do?sysparm_article=KB0099390) on Copado Best Practice and Quality Gate. When you submit a PR, it will compare your feature branch with upstream branch (where the destructive change will take place).

The below is a screenshot of the PR of an incorrect destructive change: If you see more files than originally expected, your base branch or the changes are wrong. Please consider the above step and re-commit the story.



The below is what a correct pull request should look like. See how the correct deletion is showing. You should always review the files before approving the Copado Quality Checks.



Special Consideration for Profile and Permission

When you are deleting / changing any metadata related to profile and permission sets, it will trigger Copado to also update all profiles and permissions with the necessary changes. This will require the PR security Approval step mentioned in the PR document above.



Revised by Regin Kanagappa
Last modified 2 months ago

Helpful?